

AIMLZG546 - Software Engineering for Machine Learning

Assignment I - Report

Group No: 13

Project: Credit Card Fraud Detection

Repository: <https://github.com/sanjaykumarpushadapu/SE-ML-Assessments>

Group Details

Sl. No	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (out of 100)
1	2025ae05576	Anirudh Anand	Report lead. Wrote the domain and problem statement, ML problem formulation, stakeholders, scope, functional requirements and measurable goals (Sections 1 and 2) and the GR4ML Business View. Compiled the final report and PDF.	25
2	2025ae05178	Aniketh Paul	Wrote the GR4ML Analytics Design View, the Data Preparation View and the top three quality requirements with justification (Sections 3.2, 3.3 and 4). Built the data part of the pipeline in the notebook: EDA, ingest, clean, features and split filters.	25
3	2025ae05898	Pushadapu Sanjay Kumar	Designed the system architecture diagram showing ML and non-ML components, selected the two patterns and wrote Sections 5 and 6. Built the FastAPI prediction service, input validation, prediction logging and the automated tests.	25
4	2025af05111	Biswajeet Mahato	Built the model part of the pipeline: train and evaluate filters, threshold selection on validation, MLflow tracking and model registry. Ran the notebook end to end, took the screenshots and wrote Sections 7.2, 7.3, 8 and 9.	25
			Total	100

Objective 1: Requirements Formulation

1. Domain and Problem Statement

1.1 Domain

The domain we chose is banking, and within it credit card fraud detection. Millions of card payments happen every day and each one has to be approved or declined in a few seconds. If a fraudulent payment goes through, the bank normally has to bear the loss and the customer also loses trust in the bank. So we felt this is a good practical area for a machine learning application.

1.2 Problem Statement

Banks lose a lot of money every year because of fraudulent credit card transactions. The fraud team cannot check every payment by hand, and manual checking is too slow to stop a payment that is being processed. Fraud is also very rare compared to genuine payments (less than 1%), so fixed rules either miss the fraud or block many genuine customers.

In this assignment we propose an application that checks each card transaction when it arrives and gives the probability that it is fraud. Transactions with a high probability are flagged, and the fraud analysts look only at the flagged ones instead of all transactions. Our goal is to catch at least 90% of the frauds, wrongly flag less than 2% of genuine transactions, and give the answer fast enough to be used during the payment.

1.3 Why Machine Learning is Needed

Most banks already have rule based checks, for example blocking a payment above some amount or from an unusual country. Someone has to write these rules by hand, and fraudsters slowly find ways around them. A rule also looks at only one or two fields at a time, and if the rules are made stricter, more genuine payments get declined.

A machine learning model can learn the fraud patterns from old transactions that are already marked as fraud or genuine. It can look at many features together and can be trained again when new fraud patterns come. The model also gives a probability instead of a plain yes or no, so the bank can pick a threshold that gives an acceptable balance between missed frauds and false alarms.

1.4 Stakeholders

Stakeholder	Interest in the system
Bank risk manager	Reduce fraud losses without increasing complaints from customers
Fraud analyst	Get a reliable score so that only suspicious transactions need manual review
Card holder (customer)	Genuine payments should not be declined
ML and development team	Build and maintain a model and service that are accurate and fast

1.5 Scope

These are inside the scope of this assignment:

- Checking a single transaction and returning a fraud label with its probability
- Providing this check through a prediction service (API)
- Training the model and storing it in a model registry
- Logging every prediction made by the service

Outside the scope are: connecting to a real payment network, sending alerts to customers, handling chargebacks and using real customer card data. We use a public anonymised dataset instead.

1.6 Formulation as a Machine Learning Problem

We treat this as a binary classification problem. Each transaction is labelled fraud (1) or genuine (0). The model takes the transaction features as input and gives the probability of fraud, and the final label is

decided by a threshold.

We use the "Credit Card Fraud Detection" dataset from Kaggle. It has card transactions made by European card holders, with 284,807 transactions of which only 492 (0.173%) are fraud. Except for Time and Amount, the features V1 to V28 are already anonymised by the dataset provider. The two classes are very unbalanced, so accuracy alone does not tell us much (a model that says everything is genuine would still be 99.8% accurate). Because of this we judge the model with recall, precision, F1 score, the area under the precision-recall curve and the confusion matrix.

For the design we picked two architectural patterns: Pipe-and-Filter for the training pipeline, and Microservices for serving the model as a separate prediction service together with a model registry. They are explained in Section 6.

1.7 Data Exploration (genuine vs fraud)

The charts below are copied from the notebook outputs (Section 2, Exploratory data analysis) by `build_report.py`. They show how rare fraud is and in which columns fraud looks different from genuine transactions.

Figure 1 - Class balance and amount. Only 492 of the 284,807 transactions (0.173%) are fraud, and most amounts are small with a long tail of large payments.

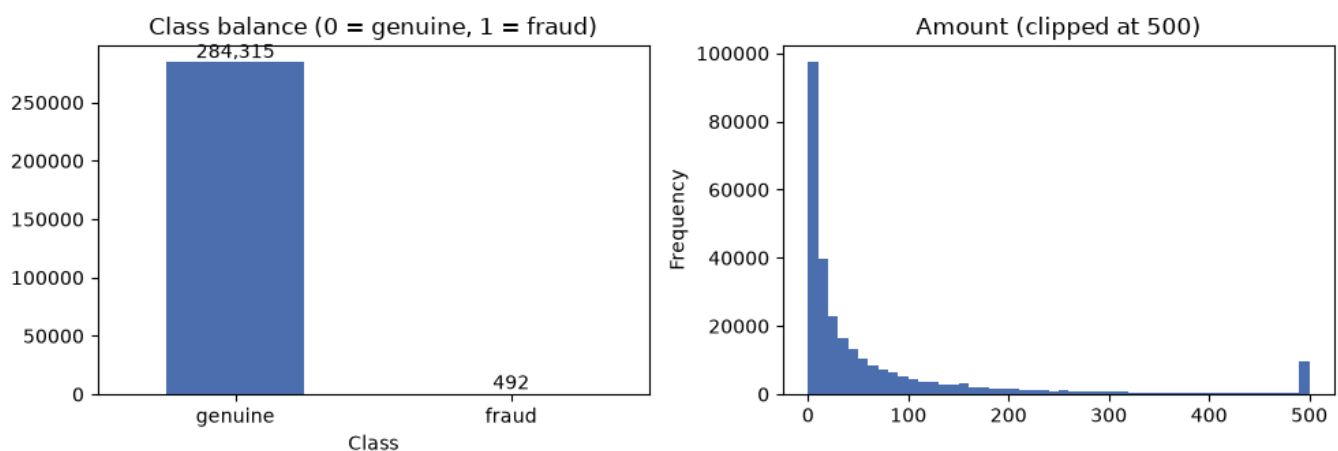


Figure 2 - Time of the transactions. Each bar is the share of that class's transactions in one hour (Time is counted from the first transaction, so hour 0 is not midnight). 27.0% of the frauds fall in hours 0

to 6, where only 9.8% of the genuine transactions happen, so fraud is relatively more common when there are few genuine payments. This is why Time is kept as a model input.

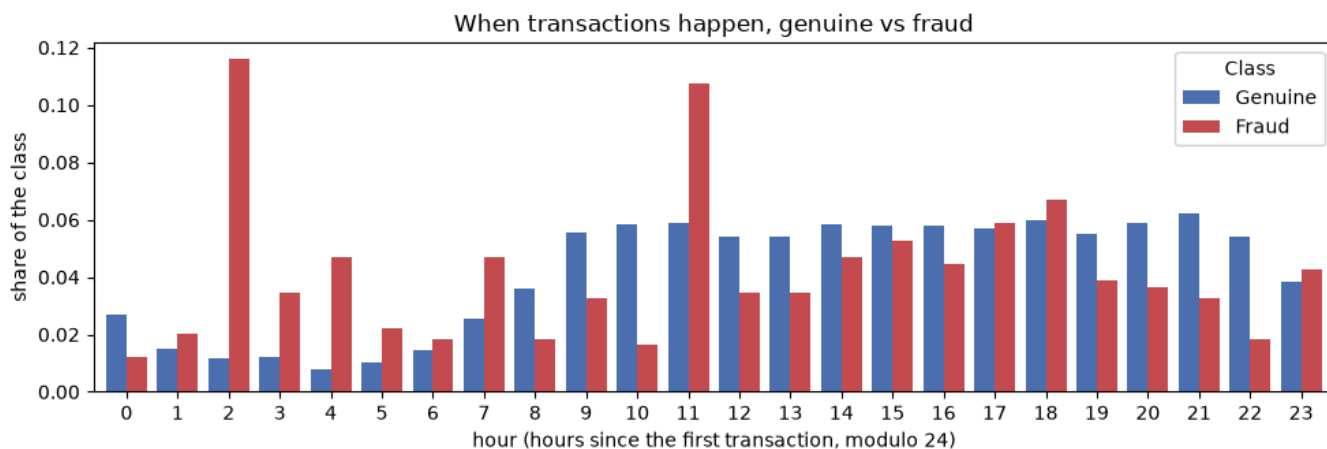


Figure 3 - Amount. The median fraud is 9.25 against 22.00 for a genuine payment, and the 95th percentile is 640.90 against 364.41, so frauds are more spread out. Amount has a long tail, with a maximum of 25,691.16, which is why it is scaled with a RobustScaler in the Data Preparation View (Section 3.3).

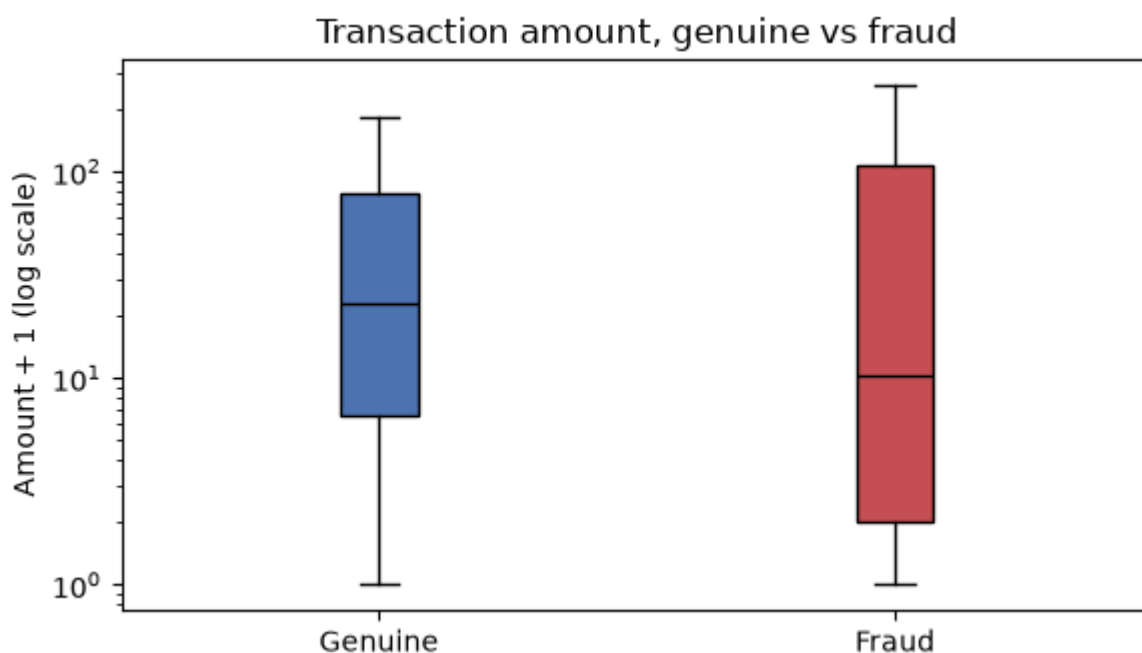
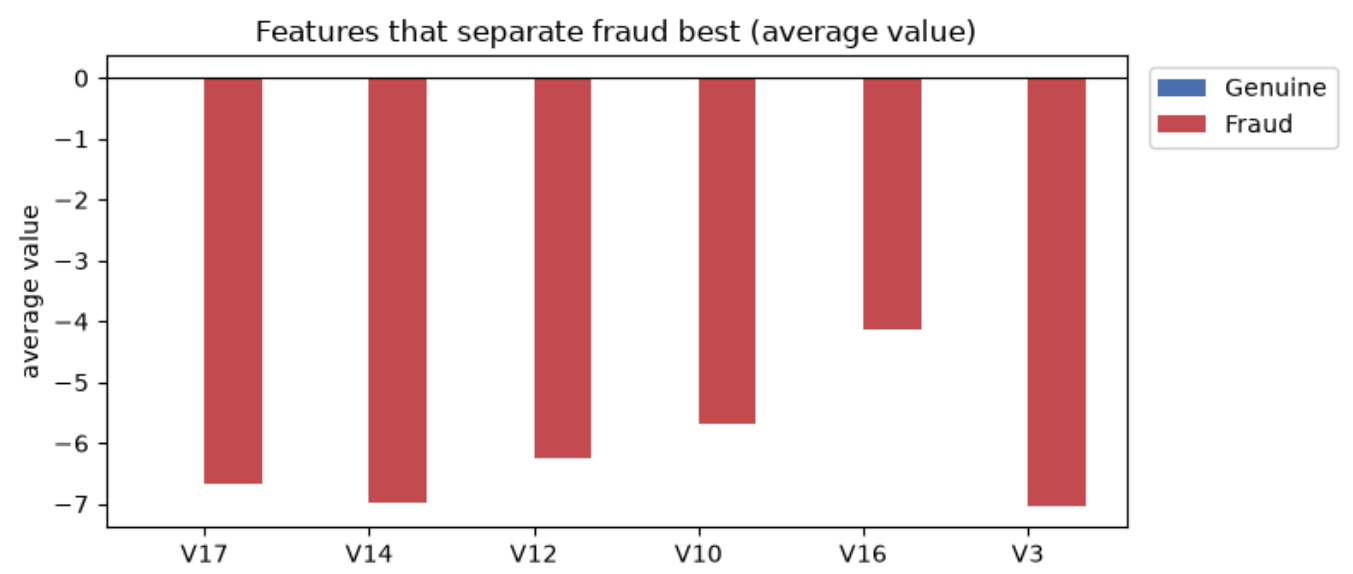


Figure 4 - Features that separate fraud best. These are the 6 V features whose average differs most between the classes, measured in standard deviations (V17 7.9, V14 7.3, V12 6.3, V10 5.2, V16 4.7, V3 4.7). The genuine averages are close to 0 and the fraud averages are far from it. A simple linear model

can use such differences, which supports Logistic Regression as a candidate in the Analytics Design View (Section 3.2).



2. Requirement Specifications and Measurable Goals

2.1 Functional requirements

We checked these requirements using the automated tests in Section 2.3 and the notebook runs. The table lists the evidence for each one.

ID	Requirement	Verified by
FR1	Detect fraud transactions: the model gives a fraud probability. We compare it with the threshold chosen on validation to decide the label.	test_BR001_label_is_probability_against_threshold, test_BR001_boundary_probability_equal_to_threshold_is_fraud; Screenshots 4 and 5
FR2	Real-time prediction: POST /predict accepts one transaction (Time, V1 to V28 and Amount) and returns its probability, label, threshold and model version. We measure the response time against goal G3.	test_BR009_response_carries_model_version; Screenshots 4 and 5; latency measurements in Section 7.4
FR3	Audit logging: the system writes one row to a prediction log for every successful prediction.	test_BR008_every_call_is_logged, test_BR008_log_failure_is_an_error_not_silent, test_BR007_invalid_input_is_not_logged; Screenshot 6
FR4	Model management: we store the trained model in MLflow. When the API starts, it looks up the champion alias and loads that model version.	test_BR009_loads_only_the_champion_alias, test_BR009_tracking_uri_can_point_to_registry_service; Screenshots 2 and 7
FR5	The training pipeline reads the dataset, cleans it, splits it, trains the model and reports the evaluation	test_BR010_run_pipeline_chains_filters_in_order, test_BR005_metrics_include_required_measures, test_BR011_same_seed_gives_same_split_and_results; Screenshot 1

ID	Requirement	Verified by
	metrics. The pipeline can be run again to retrain.	
FR6	The system rejects invalid input (missing field, wrong type, NaN or infinity, negative Amount, unknown field) with HTTP 422 and does not return a prediction for it.	test_BR007_missing_field_rejected, ..._negative_amount_rejected, ..._wrong_type_rejected, ..._extra_field_rejected, ..._nan_and_inf_rejected; invalid-request cell of the notebook
FR7	The service has a health endpoint that reports whether it is running and which model version it serves.	test_BR009_response_carries_model_version (calls /health); Docker Compose health checks (Screenshot 7)

Requirement traceability:

The following table shows which part of our code handles each main requirement.

Requirement	Responsible component	Implementation
FR1 Detect Fraud Transactions	ML model loaded by the prediction service	pipeline/train.py trains the model; api/main.py computes the probability and applies the threshold.
FR2 Real-Time Prediction	FastAPI API	api/main.py exposes POST /predict and returns the result.
FR3 Audit Logging	Logging module	api/prediction_log.py appends a CSV row for each successful prediction.
FR4 Model Management	MLflow model registry	pipeline/registry.py registers the champion; api/main.py loads its fixed version and threshold.

2.2 Measurable goals

ID	Goal	GR4ML concept	Metric	Target	Result (latest run)	Met?
G1	Catch fraud	Indicator of the strategic goal; softgoal High recall	Recall on the test set	at least 90%	86.3%	No
G2	Avoid blocking genuine customers	Indicator of the strategic goal; softgoal Few false alarms	False-positive rate on the test set	below 2%	1.91%	Yes
G3	Decide while the payment is processed	Softgoal Low latency	p95 latency of the prediction API	below 200 ms	27.8 ms	Yes
G4	Reduce losses	Strategic goal	Share of fraud money caught (test set)	at least 30%	82.4%	Yes

Model-quality results use the verified registry evidence identified in Section 7; latency uses the saved native API measurements. Historical notebook outputs are labelled separately. If a target is not met, we report the real value and do not tune anything on the test set.

Latest acceptance status: G1 fails: test recall is 86.32% against the 90% target. The model caught 82 of 95 frauds and missed 13. The 95% exact binomial interval is 77.74% to 92.51%; it does not change the pass/fail decision. Passing software tests does not establish model-quality acceptance. The reused test set is exploratory, and a fresh untouched hold-out is needed before claiming production acceptance.

Earlier baseline: Docker version 14 caught 81 of 95 frauds: recall 85.26%, FPR 1.08%. Its 95% exact binomial recall interval was 76.51% to 91.70%, which did not make G1 pass. The current result is reported above. Test-set threshold trade-offs remain post-hoc analysis, not inputs to model selection or promotion.

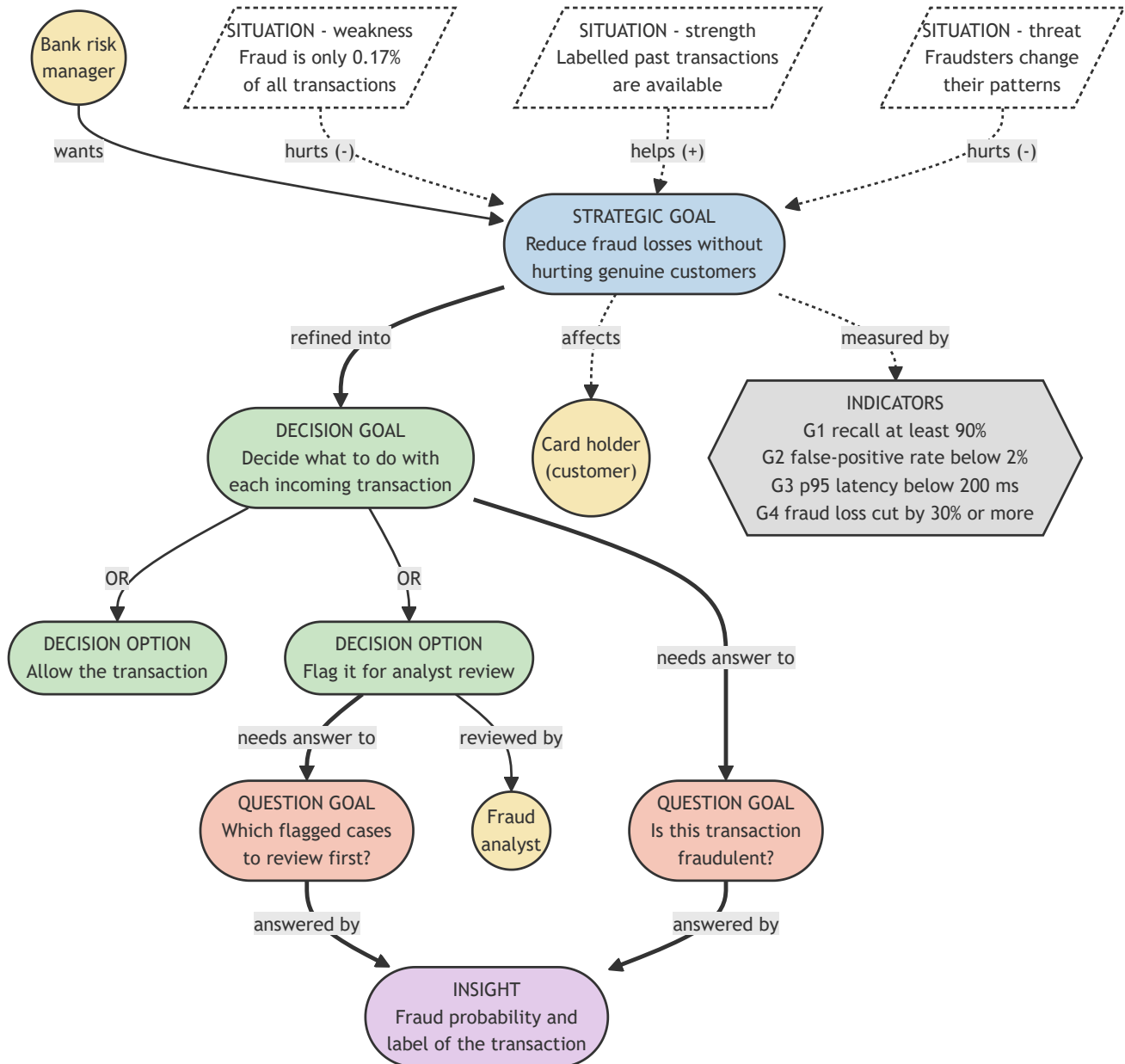
To improve recall, investigate candidate models, class weighting and threshold selection using training and validation data only. Freeze the selected pipeline and acceptance criteria before evaluating on fresh, untouched hold-out data. The already-inspected test set must not become a threshold-tuning or model-selection set.

2.3 Business rules

ID	Rule
BR-001	A transaction is labelled fraud only if its fraud probability is greater than or equal to the threshold.
BR-002	The threshold is chosen on validation, aiming for 95% recall within FPR below 2%. If infeasible, we use the highest feasible recall at or above 90%. The settings and threshold are stored in MLflow.
BR-003	The scaler is fitted on the training data only.
BR-004	The scaling is saved inside the model pipeline and applied the same way at serving time.
BR-005	Evaluation reports recall, precision, F1, PR-AUC, ROC-AUC, false-positive rate and the confusion matrix.
BR-006	Targets are compared honestly with the test results; nothing is tuned on the test set.
BR-007	Invalid input is rejected with HTTP 422 and is not logged.
BR-008	One log row is written per prediction; a failed write is reported as an error.
BR-009	The API loads only the model with the alias champion.
BR-010	Training runs as a chain of six filters: ingest, clean, features, split, train, evaluate.
BR-011	A fixed seed (42) and a single config make the run reproducible.

3. GR4ML Views

3.1 Business View

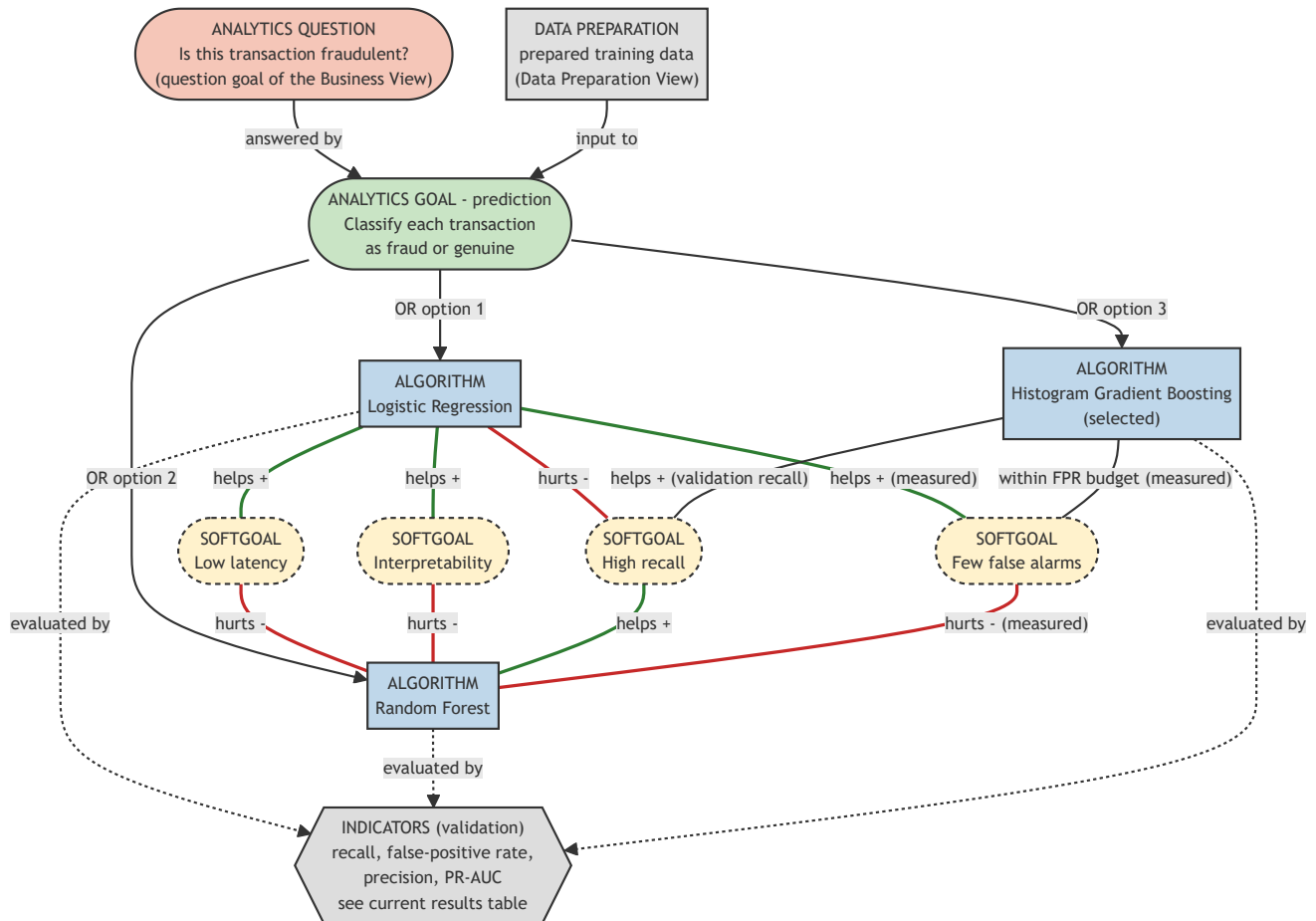


Colour key: yellow circles are actors. Blue, green, red and purple rounded boxes are the strategic goal, the decision goal with its options, the question goals and the insight. The grey hexagon holds the indicators, and the dashed white parallelograms are situations. Thick arrows connect the goals, thin arrows show OR alternatives and who acts, and dotted lines are influence links (helps +, hurts -), "affects" and "measured by".

Explanation: The bank risk manager wants the strategic goal, which is to reduce fraud losses without hurting genuine customers. The card holder is affected because a wrongly blocked payment is annoying for them. The strategic goal is refined into one decision goal: decide what to do with each incoming transaction. This decision has two alternatives (an OR refinement): allow the transaction, or flag it for review by a fraud analyst. To take the decision the bank needs the answer to the question goal "is this transaction fraudulent?", and for the flagged cases the analyst also needs to know which ones to review

first. Both questions are answered by the same insight, the fraud probability and label of the transaction (a higher probability is reviewed first). Three situations influence the strategic goal. Fraud is very rare (weakness), so it is hard to learn and easy to miss. Labelled past transactions exist (strength), so a supervised model can be trained. Fraudsters change their patterns (threat), so the model must be retrained, which the pipeline and the registry make possible. The strategic goal is measured by the four indicators G1 to G4, which are the measurable goals in Section 2.2.

3.2 Analytics Design View



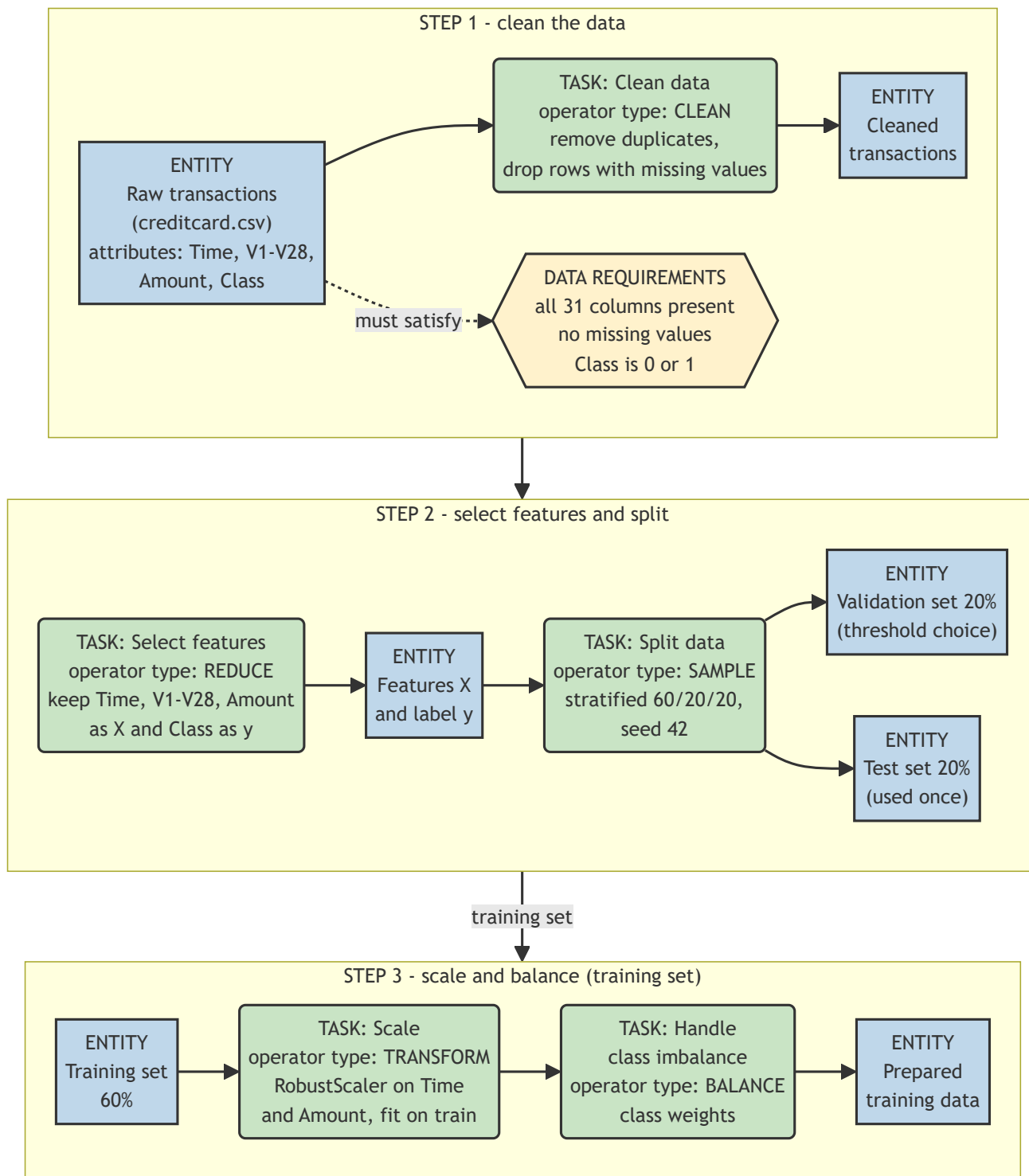
Colour key: green links are positive contributions (+) and red links are negative contributions (-). The Logistic Regression and Random Forest contribution links describe the earlier comparison shown in Figure 6. The boosted candidate's measured links use the current validation results table. The other links show our expectations before training.

Explanation: The analytics question comes from the Business View: is this transaction fraudulent? It is answered by a prediction goal that classifies each transaction as fraud or genuine. We first compared Logistic Regression and Random Forest, then added Histogram Gradient Boosting to investigate the recall shortfall. We expected the linear model to be easier to explain and the tree models to learn more complex patterns. All three use the same prepared training data and validation indicators. We now aim for a recall margin while keeping the false-positive limit; the measured choice is reported below, rather than decided from the test scores.

We compared Logistic Regression, Random Forest and Histogram Gradient Boosting. Histogram Gradient Boosting was selected using validation recall and the false-positive limit, not test results. It

reached 92.6% recall and 1.84% FPR on validation, then 86.3% recall and 1.91% FPR in the exploratory test evaluation.

3.3 Data Preparation View



Colour key: blue boxes are entities (data) with their attributes, green boxes are data preparation tasks, and each task names its GR4ML operator type (CLEAN, REDUCE, SAMPLE, TRANSFORM, BALANCE) and how we applied it. The yellow hexagon lists the data requirements the raw data must meet. Arrows show the flow of the data.

Explanation: The raw entity has Time, V1 to V28, Amount and Class; ingest requires all 31 columns. The CLEAN task drops 1,081 duplicate rows and rows with missing values. It then rejects an empty dataset, nonnumeric or infinite features, negative Amount, and labels other than 0 or 1; both classes must remain. The REDUCE task selects the 30 inputs and label. The SAMPLE task uses a stratified 60/20/20 split with seed 42. On training data only, TRANSFORM fits the RobustScaler for Time and Amount, and BALANCE supplies class weights to the classifier. Validation selects the threshold; the frozen pipeline is evaluated on the test set. The API also enforces finite features and nonnegative Amount and rejects extra fields.

3.4 GR4ML concept mapping

GR4ML concept	In this project
Actors	Bank risk manager, fraud analyst, card holder
Strategic goal	Reduce fraud losses without hurting genuine customers
Decision goal	Decide what to do with each transaction (OR: allow, or flag for analyst review)
Question goals	Is this transaction fraudulent? Which flagged cases to review first?
Insight	Fraud probability and label per transaction
Situations	Fraud is rare (weakness), labelled history exists (strength), fraud patterns change (threat)
Indicators	G1 recall, G2 false-positive rate, G3 p95 latency, G4 fraud loss reduction (Section 2.2)
Analytics question / goal	Is this transaction fraudulent? / prediction (binary classification)
Algorithms	Logistic Regression, Random Forest, Histogram Gradient Boosting (selected in the current run)
Softgoals	High recall, few false alarms, low latency, interpretability
Data preparation entities	Raw transactions, cleaned data, features and label, train/validation/test sets, prepared data
Tasks and operator types	Clean (CLEAN), select features (REDUCE), split (SAMPLE), scale (TRANSFORM), class weights (BALANCE)
Data requirements	All 31 columns present, no missing values, Class is 0 or 1

4. Top Three Quality Requirements

#	Quality requirement	Measurable target	Justification (why it matters here)	How architecture supports it
1	Accuracy (recall)	Recall at least 90% with false-positive rate below 2%	A fraud that is not detected is a direct loss for the bank. The classes are very unbalanced, so overall accuracy would hide the missed frauds.	We use class weights and a separate validation set to choose the threshold, and we report recall, precision, F1, PR-AUC and the confusion matrix instead of accuracy.
2	Performance (latency)	p95 latency below 200 ms	The decision has to be taken while the payment is being processed, so a late answer is as bad as no answer.	The model is loaded once when the service starts and kept in memory. The prediction service is separate from training, so for each request it only validates the input and calls the model once.
3	Security and privacy	No card number accepted or stored; invalid input rejected	Card data is sensitive, and a service that accepts any input can be misused.	The API accepts only the numeric model features (no card number or name) and rejects any extra field. The prediction log stores only the probability, label, latency and model version.

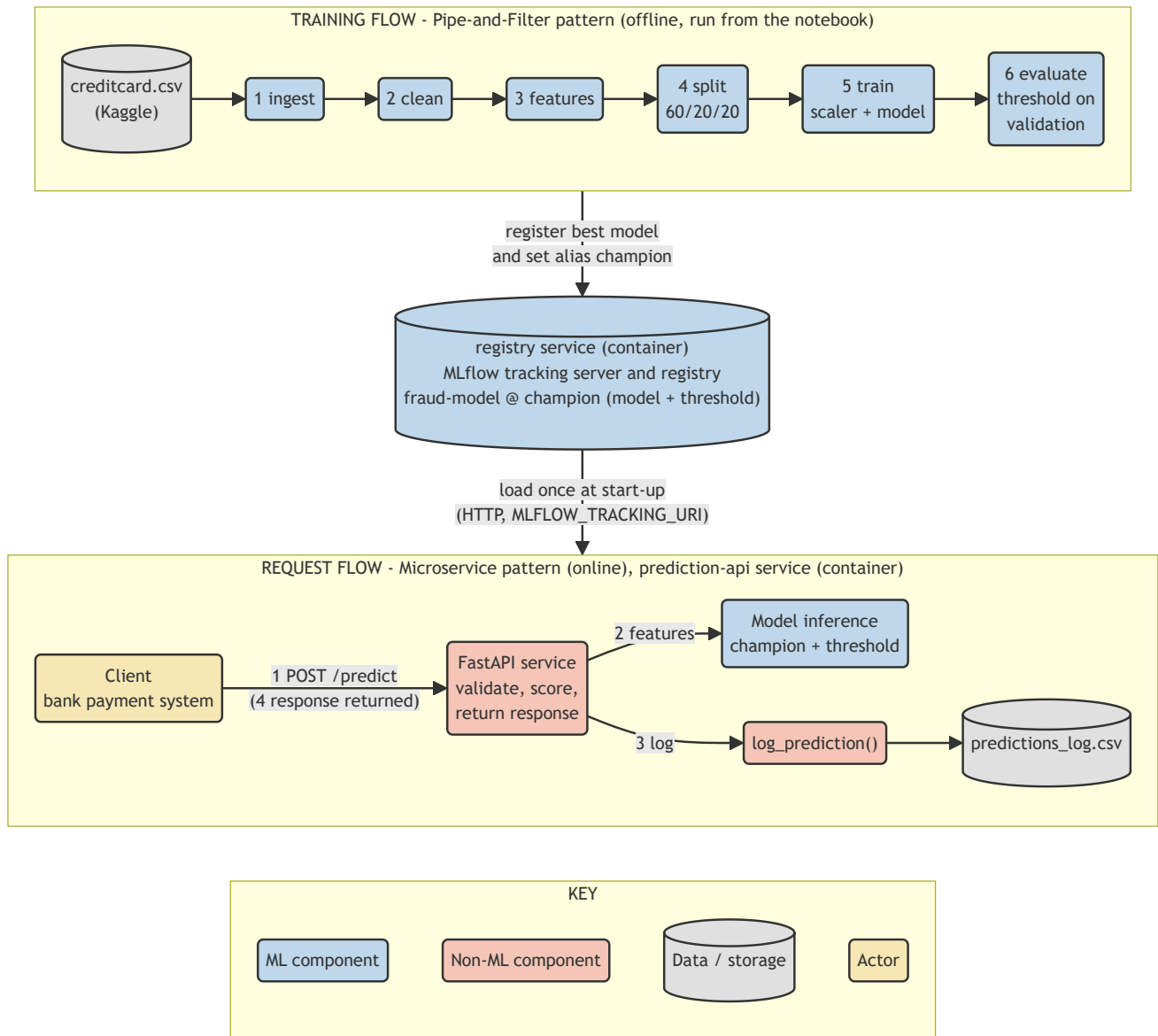
Why these three: each one is tied to a stakeholder and a measurable goal. Recall protects the bank's money (risk manager, G1 and G4), latency is needed because the answer must arrive while the payment is being processed (payment system, G3), and security and privacy protect the card holder's data. We also considered other quality requirements and ranked them lower for this application:

- **Explainability:** useful for fraud analysts, but V1 to V28 are anonymised components, so a feature-level explanation would not mean much to an analyst. We kept interpretability as a softgoal in the Analytics Design View instead.
- **Fairness:** it cannot be measured here, because the dataset has no customer attributes such as age or gender.
- **Availability and scalability:** important in production, but our prototype runs one instance of each service. The microservice design (Section 6.2) allows the prediction service to be scaled separately later.
- **Maintainability:** supported by the Pipe-and-Filter design and the 30+ automated tests, but it does not directly affect the bank's losses or the customer, so it is not in the top three.

Authentication of API clients is not part of this assignment. We list it as a limitation of our prototype.

Objective 2: System Architecture

5. System Architecture Diagram



The key inside the diagram explains the colours: blue is an ML component, red is a non-ML component, grey is data or storage and yellow is an actor.

ML components: the six pipeline filters (ingest, clean, features, split, train, evaluate), the MLflow model registry that keeps the trained model and its threshold, and the model inference step inside the prediction service.

Non-ML components: the FastAPI service with the /predict and /health endpoints, the pydantic request validation, the log_prediction function with the prediction log file, and the client that sends the transactions.

Request flow: (1) The client sends a transaction to /predict. (2) The FastAPI service validates it and gives the features to the loaded champion model, which returns a probability. (3) The service compares

the probability with the threshold and writes one row to the prediction log through `log_prediction()`, and (4) the response goes back to the client.

Deployment: with Docker Compose the registry and the prediction API run as two separate containers that talk over HTTP (Section 7.7).

Training flow: the Kaggle dataset goes through the six filters. In the last filter the threshold is chosen on the validation set, the results are logged to MLflow, and the best model is registered as `fraud-model` with the alias `champion`. The prediction service loads this model when it starts.

6. Architectural Patterns

6.1 Pattern 1: Pipe-and-Filter

Why we chose it: Our training workflow has several steps: reading the data, cleaning it, preparing features, splitting it, training and evaluation. Putting all of them in one script would make it harder to test or change one step. Pipe-and-Filter lets each step take the previous step's output and pass its result to the next one.

How we used it: We kept the six stages in separate modules under [pipeline](#). [pipeline/run_pipeline.py](#) calls them in order. The training stage accepts a classifier name, so we could compare Logistic Regression, Random Forest and Histogram Gradient Boosting without changing the earlier stages.

Benefits and limits: We can test each stage separately, reuse the data-preparation code across the models, and change one stage without rewriting the whole pipeline. We still pass complete tables in memory. This works for our dataset, but we have not tested it with much larger data.

6.2 Pattern 2: Microservices

Why we chose it: Training takes more resources and runs only when we need a new model. The prediction API needs to answer requests without training a model each time. Keeping these jobs separate makes deployment and maintenance easier.

How we used it: Training runs as an offline job. We run MLflow and FastAPI in two separate containers using [docker-compose.yml](#). MLflow stores the models, and [api/main.py](#) loads `fraud-model` through the `champion` alias at startup. The API handles `/predict` and `/health`; it does not train. It contacts the registry over HTTP, and Docker waits for the registry's health check before starting the API (Section 7.7, Screenshot 7).

Benefits and limits: We can restart the API without running training again, and the model version is included in each response. The design could support more API instances, but we have not tested scaling or failure recovery. Both containers use the same host and project folder, so a host or storage failure can affect both. The API also needs the registry when it starts, and it must be restarted to load a new champion. For a production setup, we would need a shared artifact store and logging that works safely across multiple API instances.

7. Implementation

7.1 Technology stack

Layer	Technology	Purpose
ML pipeline	Python, pandas, scikit-learn	Filters for data preparation, training and evaluation
Model registry	MLflow (SQLite backend)	Tracking of runs and registry of the champion model
Prediction service	FastAPI, pydantic, uvicorn	Separate prediction microservice with input validation
Logging	CSV file written by log_prediction()	One row per prediction
Testing	pytest, FastAPI TestClient	Tests named after the business rules
Deployment	Docker, Docker Compose	Registry and prediction API as two containers
Runtime	Jupyter notebook 13.ipynb (runs in Google Colab or VS Code)	Runs everything from top to bottom, no GPU needed

7.2 Pipe-and-Filter pipeline

The notebook writes each filter to its own file and run_pipeline.py joins them. ingest checks that the file and the columns exist, and stops with an error if they do not. clean drops duplicate rows and rows with missing values. features takes Time, V1 to V28 and Amount as inputs and Class as the label. split does a stratified 60/20/20 split with seed 42. train fits a scikit-learn Pipeline where a RobustScaler for Time and Amount comes before the classifier, so the scaler is fitted on the training set only and is saved together with the model. evaluate aims for 95% validation recall within FPR below 2%; when that margin is infeasible, it takes the highest feasible recall at or above 90%. The model and threshold are frozen before test scoring. The repeated test evaluation is exploratory because we already saw the earlier results.

```
# each stage hands its output to the next one
raw = ingest(data_path)
df = clean(raw)
X, y = build_features(df)
parts = split(X, y)
# train every candidate on the training set and judge it on the validation set
fitted = {}
for name in candidates:
    model = train(parts["X_train"], parts["y_train"], name)
    threshold, val_m = evaluate_validation(model, parts["X_val"], parts["y_val"])
    fitted[name] = (model, threshold, val_m)
# the winner is chosen from validation results only; the test set is used once at the end
winner = pick_winner(fitted) # validation recall margin within the false-positive budget
model, threshold, val_m = fitted[winner]
test_m = evaluate_test(model, parts["X_test"], parts["y_test"], threshold)
```

About these results: The model results and Docker screenshots use Docker champion version 17. The latency measurements and charts are from the saved native notebook run using version 8. The native and Docker registries number their versions separately. We checked the Docker version, threshold and results against the running API and the model's MLflow run.

Model results: The dataset has 284,807 rows with 492 frauds (0.173%). After removing 1,081 duplicate rows, 283,726 rows remain with 473 frauds (0.167%). The stratified split gives 170,235 training, 56,745 validation and 56,746 test rows. We aimed for 95.0% recall on validation while keeping FPR below 2.0%. Where that was not possible, we chose the highest validation recall within the FPR limit, provided it reached the 90.0% minimum. Infeasible candidates are shown for comparison.

Model (validation)	Threshold	Recall	Precision	FPR	PR-AUC
Logistic Regression	0.7692	0.9043	0.1286	1.02%	0.786
Random Forest	0.0010	0.9574	0.0067	23.59%	0.859
Histogram Gradient Boosting	0.0226	0.9255	0.0769	1.84%	0.858

We compared 3 models on validation. We selected Histogram Gradient Boosting (version 17) using validation results only: it gave the highest feasible recall (92.6%) within the false-positive limit (1.84%). Below are the test results at the frozen validation-selected threshold:

Recall	Precision	F1	PR-AUC	ROC-AUC	FPR
0.8632	0.0705	0.1304	0.7796	0.9716	1.91%

Confusion matrix on the test set: TN 55,570, FP 1,081, FN 13, TP 82. **The FPR target (below 2%) is met, but the recall target (at least 90%) is not met on the test set (86.3%).** On validation the recall was 92.6%. The test set has only 95 frauds and the validation set 94, so missing just one more fraud changes recall by about one percentage point, and a threshold that is only just enough on validation can fall short on new data. We report the result as it is and did not tune the threshold on the test set.

Fraud loss (goal G4): We measured the loss in money using the Amount column and assumed that every flagged fraud is stopped. The 95 frauds in the test set add up to 14,766.31 and the model catches 12,167.76 of it, so the fraud loss goes down by 82.4%, against the target of 30% (met). The cost is that genuine transactions worth 280,972.10 were also flagged, and a real bank would have to consider the cost of these blocked payments too. This is only an estimate from the test set and it does not include chargeback fees or the cost of reviewing the flagged transactions.

Note on the selection rule: After seeing the earlier recall shortfall, we added a boosted-tree candidate and a validation recall margin. We compared two boosted-tree configurations using training and validation data, then froze the choice before this test evaluation. The test set had already been inspected, so this is an exploratory result, not a fresh hold-out assessment.

Figure 5 - Confusion matrix on the test set. This is the saved native notebook chart for model version 8, not the current Docker model. The current model's counts are in the results table above.

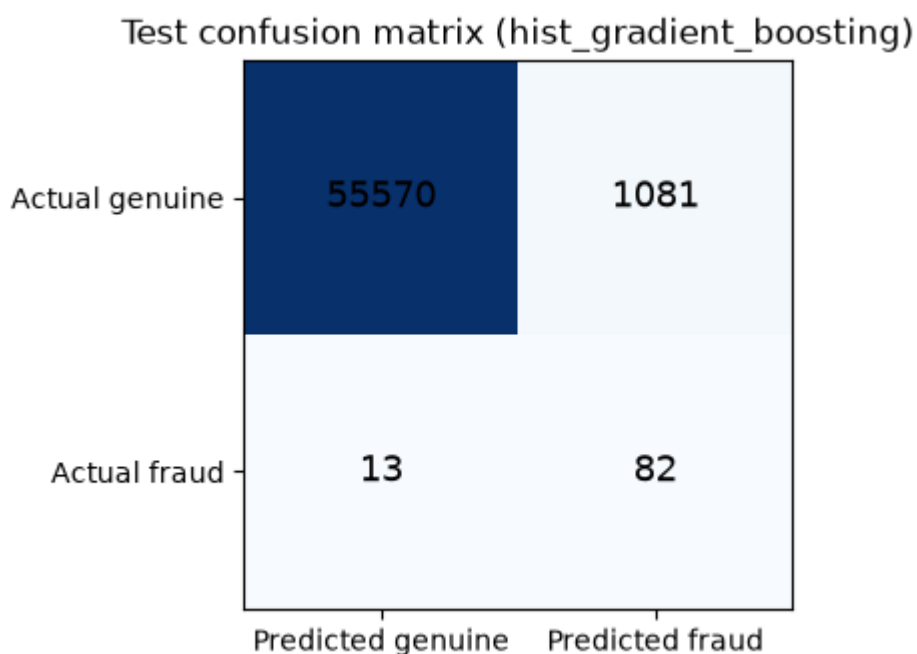
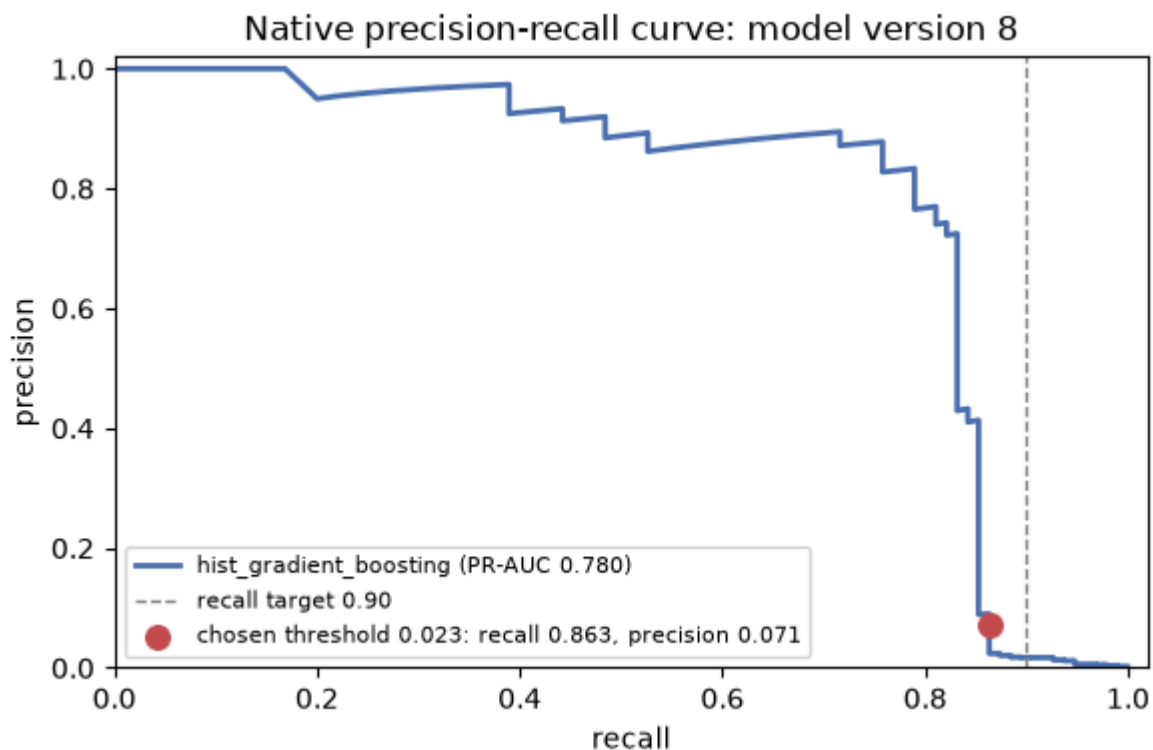


Figure 6 - Model comparison on the validation set. This saved native table shows all candidate validation results and the winner's test result for native model version 8. The test row reports the frozen winner; it was not used to select the model. Docker model results are reported separately above.

	threshold	recall	precision	f1	pr_auc	roc_auc	fpr
hist_gradient_boosting (validation)	0.0226	0.9255	0.0769	0.1419	0.8584	0.9757	0.0184
random_forest (validation)	0.0010	0.9574	0.0067	0.0133	0.8592	0.9662	0.2359
logistic_regression (validation)	0.7692	0.9043	0.1286	0.2252	0.7860	0.9790	0.0102
hist_gradient_boosting (TEST, winner)	0.0226	0.8632	0.0705	0.1304	0.7796	0.9716	0.0191

Figure 7 - Precision-recall curve on the test set. This is the saved notebook's frozen native-model curve, not a re-evaluation of the Docker model. The red point uses its validation-selected threshold; the dashed line is the 90% recall target. Post-hoc analysis of this saved test set gives precision 1.7% at the

recall target, with 4,886 genuine transactions flagged. This analysis is not used to tune, serve or promote a model.



7.3 MLflow model registry

Every candidate model is logged as an MLflow run with its parameters (model name, seed, threshold) and its validation metrics. In demonstration mode, the winning model also gets its test metrics, is registered as fraud-model, and the alias champion is set on this version. The threshold is saved as a run parameter, so the prediction service reads the same value that was chosen during evaluation and it is not written inside the service code. We stored Histogram Gradient Boosting as fraud-model version 17 in the Docker registry, with the alias champion and threshold 0.0226. When the API starts, it looks up champion once and loads that version and its threshold. This keeps the model and threshold together even if champion changes while the API is starting. We need to restart the API to load a new champion. This assignment uses explicit demonstration mode, so the served champion is not a production acceptance claim. New runs record model_quality_acceptance and deployment_scope tags. run_pipeline(demonstration=False) refuses to promote a winner with missing or failed recall, false-positive or fraud-loss checks. Passing that gate still requires separate fresh-holdout and service validation.

7.4 Prediction microservice (FastAPI)

The service has two endpoints. GET /health returns the status and the model version. POST /predict takes the 30 input values and returns request_id, probability, is_fraud, threshold, model_version and latency_ms. The label is true only when the probability is greater than or equal to the threshold. Invalid input gets HTTP 422 and is not logged. If the prediction log cannot be written, the call fails with HTTP 500 and does not continue silently.

```

@app.post("/predict", response_model=Prediction)
def predict(tx: Transaction) -> Prediction:
    """Score one transaction; the label is probability >= threshold and nothing else (BR-001)."""
    start = time.perf_counter()
    # Invalid requests never reach this point, FastAPI has already answered with 422.
    row = pd.DataFrame([tx.model_dump()], columns=config.FEATURES)
    # The saved model scales the input itself, so no separate preprocessing is done here (BR-004).
    probability = float(app.state.model.predict_proba(row)[0, 1])
    is_fraud = probability >= app.state.threshold
    latency_ms = (time.perf_counter() - start) * 1000
    request_id = str(uuid.uuid4()) # unique id so a log row can be matched to a call
    try:
        log_prediction(request_id, probability, is_fraud, latency_ms, app.state.version)
    except OSError as exc:
        # A failed log write is reported as an error, never ignored (BR-008).
        raise HTTPException(status_code=500, detail=f"Prediction log failed: {exc}") from exc
    return Prediction(request_id=request_id, probability=probability, is_fraud=is_fraud,
                      threshold=app.state.threshold, model_version=app.state.version,
                      latency_ms=round(latency_ms, 3))

```

Response time: In the saved native notebook run (model version 8), we sent 100 requests. The median was 13.3 ms, the p95 was 27.8 ms and the maximum was 30.2 ms. The p95 target of below 200 ms was met.

7.5 Prediction logging

Every successful prediction is added to predictions_log.csv with the time, request id, probability, label, latency and model version. We do not write the feature values to the log, so no transaction data is stored there.

```

def log_prediction(request_id: str, probability: float, is_fraud: bool, latency_ms: float,
                  model_version: str, path: Path | None = None) -> None:
    """Write one log row; an OSError is raised if the file cannot be written."""
    path = Path(path or config.LOG_PATH)
    # Only result data is logged. The transaction values themselves are not stored.
    row = {"timestamp": datetime.now(timezone.utc).isoformat(), "request_id": request_id,
          "probability": round(probability, 6), "is_fraud": int(is_fraud),
          "latency_ms": round(latency_ms, 3), "model_version": model_version}
    with _lock:
        new_file = not path.exists() or path.stat().st_size == 0 # write the header once
        with path.open("a", newline="") as f: # "a" = append, old rows are kept
            writer = csv.DictWriter(f, fieldnames=FIELDS)
            if new_file:
                writer.writeheader()
            writer.writerow(row)

```

7.6 End-to-end run and test calls

We run the notebook from top to bottom after restarting the runtime. It unzips the dataset from data/creditcard.zip (in Colab it asks for an upload), runs the pipeline, shows the MLflow registry and starts the service in the background. Then it sends one fraud row and one genuine row from the test set, three invalid requests (negative Amount, missing field, unknown field) and 100 requests to measure the latency. At the end it shows the prediction log, runs the automated tests with pytest and, when

Docker is installed, starts the two containers and calls the API through them. In the saved notebook test run, **58 tests passed**.

7.7 Containerised deployment (Docker Compose)

The API and model registry run as separate Docker services:

- **prediction-api**: the FastAPI service (container port 8000), using `MLFLOW_TRACKING_URI=http://registry:5000` and `MLFLOW_ARTIFACT_DIR=/app/mlruns`.
- **registry**: the MLflow tracking and registry service (container port 5000). It runs SQLite at `/app/docker-mlflow.db`. SQLite is embedded, not a separate database-server container.

Both services mount the project at `/app`, independent of its Windows, macOS or Linux host path. The database persists as `docker-mlflow.db`; Docker model files persist in `mlruns/`. These files survive stopping and removing containers. Native Python uses separate `mlflow.db` and `mlruns-native/`. The native reset preserves Docker artifacts and the shared prediction log. Evaluation outputs in `artifacts/` are shared, so train native and Docker workflows sequentially.

Docker chooses available loopback host ports instead of requiring host ports 5000 and 8000. The final cell reads these ports from Compose and prints the actual URLs. It starts the registry, trains and registers a model inside Docker, recreates the API to load the selected champion version, verifies requests, and leaves both services running. They use `restart: unless-stopped`.

This retrains all three candidates and requires the real dataset. Docker must be installed and running. Output uses UTF-8 decoding with replacement for invalid bytes. Failed checks leave services available for inspection with `docker compose logs`.

This is an assignment demonstration, not an accepted production deployment. The runner defaults to `demonstration=True`: a winner that misses a goal can still be served, but new runs are tagged `deployment_scope=demonstration_only` and retain their real acceptance status. `run_pipeline(demonstration=False)` rejects promotion when recall, false-positive or fraud-loss checks fail. Strict rejection leaves any existing champion unchanged. Fresh untouched evaluation and service-quality checks are still needed for production.

To start or retrain from the project folder:

```
docker compose up -d --build --wait registry
docker compose run --rm prediction-api python -m pipeline.run_pipeline
docker compose up -d --no-deps --force-recreate --wait prediction-api
docker compose port prediction-api 8000
docker compose port registry 5000
```

Open `http://<prediction-api-address>/docs` and `http://<registry-address>` using the printed addresses. Resume a trained deployment with `docker compose up -d --wait`. Stop it explicitly with `docker compose down`; database and model files remain in the project folder.

8. Application Screenshots

Verification safeguards: Native results are read from the native registry's current champion, not the metrics file shared with Docker. The frozen model must reproduce its logged validation and test metrics;

registry, version and run ID accompany the results. A single version-pinned scorer verifies the current champion, threshold and reproduced metrics and stops on failure. Plotting checks registry, run, version, threshold, row count and checksum and labels the curve as native. The Docker notebook cell checks API/registry/metrics agreement, both sample predictions, and HTTP 422 for negative Amount, missing features and extra fields. Manual report generation verifies current metrics against the live Docker API and champion's logged run; keep Docker services running when building its report. Older saved notebook outputs are explicitly labelled historical and do not block verified current results. Save the latest notebook outputs to refresh its charts and other recorded evidence. Native and Docker version numbers are independent and must not be substituted for each other. Shared training outputs still require sequential training runs.

Screenshot 1 - Pipeline run. The six pipeline stages read 284,807 rows, removed 1,081 duplicates and made the 60/20/20 split. We trained 3 models and chose Histogram Gradient Boosting. Its test recall was 0.863 and FPR was 0.0191.

Pipeline run - six-filter output

```
[1/6 ingest] rows=284807 columns=31
[2/6 clean] rows=283726 (removed 1081), fraud=473 (0.167%)
[3/6 features] X=(283726, 30)
[4/6 split] train=170235 val=56745 test=56746
[5/6 train] logistic_regression val recall=0.904 precision=0.129 pr_auc=0.7860 threshold=0.7692
[5/6 train] random_forest val recall=0.957 precision=0.007 pr_auc=0.8592 threshold=0.0010
[5/6 train] hist_gradient_boosting val recall=0.926 precision=0.077 pr_auc=0.8584 threshold=0.0226
[6/6 evaluate] winner=hist_gradient_boosting test recall=0.863 precision=0.071 f1=0.130 pr_auc=0.7796 fpr=0.0191
```

Evaluation: Docker registry, version 17

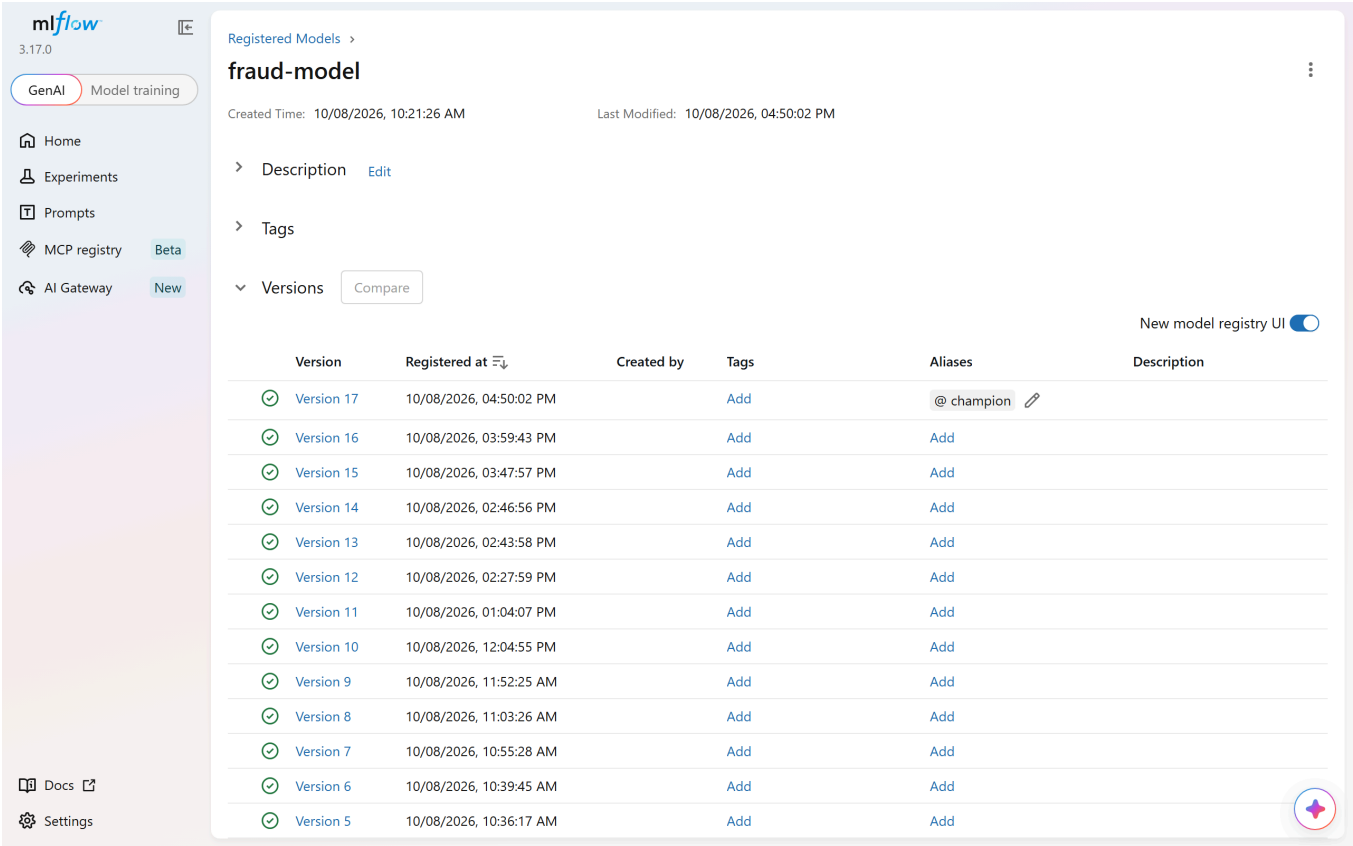
Champion: Histogram Gradient Boosting | Threshold: 0.022607

Metric	Validation	Test
Recall	0.9255	0.8632
Precision	0.0769	0.0705
PR-AUC	0.8584	0.7796
False-positive rate	0.0184	0.0191

Test recall target >= 90%: NOT MET. False-positive rate target < 2%: PASS.

Current metrics verified against live Docker; saved stage output is historical.

Screenshot 2 - MLflow. The running Docker registry shows fraud-model version 17 with the alias champion.



Screenshot 3 - FastAPI. We ran GET /health from the Docker API /docs page. The response shows model version 17 and its threshold. The number in the Swagger page heading is the API release, not the model version.

Credit Card Fraud Detection Service

0.1.0

OAS 3.1

/openapi.json

default

GET /health Health

Report that the service is up and which model version it serves.

Parameters

No parameters

Execute Clear

Responses

Curl

curl -X 'GET' \
'http://127.0.0.1:54170/health' \
-H 'accept: */*'

Request URL

http://127.0.0.1:54170/health

Server response

Code Details

200

Response body

{
 "status": "ok",
 "model": "fraud-model",
 "model_version": "17",
 "threshold": 0.022606854478447718
}

Download

Response headers

content-length: 91
content-type: application/json
date: Thu, 08 Oct 2026 11:20:46 GMT
server: uvicorn

Screenshot 4 - Fraud test call. We sent this request to the running Docker API through Swagger /docs. The API used model version 17 and returned fraud probability 0.9994, is_fraud=true, in 22 ms.

Request URL

http://127.0.0.1:54170/predict

Server response

Code	Details
200	Response body <pre>{ "request_id": "7bb8cc06-cca2-4dee-98ca-f64e9ac1c260", "probability": 0.9994017368367172, "is_fraud": true, "threshold": 0.022606854478447718, "model_version": "17", "latency_ms": 22.16 }</pre> Response headers <pre>content-length: 175 content-type: application/json date: Thu, 08 Oct 2026 11:20:46 GMT server: uvicorn</pre>

Screenshot 5 - Genuine test call. We sent this request to the running Docker API through Swagger /docs. Model version 17 returned probability 0.000, below the threshold 0.0226, so is_fraud=false.

Request URL

http://127.0.0.1:54170/predict

Server response

Code	Details
200	Response body <pre>{ "request_id": "0b26a392-0afa-4caa-bd51-97371fc06c4e", "probability": 0.00030342745551724734, "is_fraud": false, "threshold": 0.022606854478447718, "model_version": "17", "latency_ms": 8.413 }</pre> Response headers <pre>content-length: 180 content-type: application/json date: Thu, 08 Oct 2026 11:20:46 GMT server: uvicorn</pre>

Screenshot 6 - Prediction log. These two log rows have the same request IDs as screenshots 4 and 5 and use Docker model version 17. The CSV had 963 rows when we took the screenshot; we show only these two rows. This log is separate from the saved native timing measurements.

Docker prediction log - request-ID-verified records

Champion model version: 17 | CSV: /app/predictions_log.csv

Total CSV rows: 963. Showing the two requests captured in Swagger.

timestamp	request_id	probability	is_fraud	latency_ms	model_version
2026-10-08T11:20:47.386103+00:00	7bb8cc06-cca2-4dee-98ca-f64e9ac1c260	0.999402	1	22.16	17
2026-10-08T11:20:47.773297+00:00	0b26a392-0afa-4caa-bd51-97371fc06c4e	0.000303	0	8.413	17

Screenshot 7 - Docker Compose. Docker Compose shows the registry and prediction API running on container ports 5000 and 8000. Their localhost ports are assigned when they start. The Docker champion is version 17. We leave both services running until we stop them manually.

Docker Compose: live status and registry verification

NAME	IMAGE	STATUS	PORTS
COMMAND	SERVICE	CREATED	
assignment-1-prediction-api-1	fraud-detection:1.0		
"uvicorn api.main:ap..."	prediction-api	37 seconds ago	Up 34 seconds (healthy) 127.0.0.1:54170->8000/tcp
assignment-1-registry-1	sha256:563f3f407a951e3023fb62b4a00a06c88aab034fe2eaa5b9d299c520732fc76c		
"mlflow server --bac..."	registry	6 hours ago	Up 6 hours (healthy) 127.0.0.1:58706->5000/tcp

Live Docker champion version: 17
MLflow run: c6e49942bdff472eb4b5adb9f9e6ebfd
Health: {'status': 'ok', 'model': 'fraud-model', 'model_version': '17', 'threshold': 0.022606854478447718}
Current evaluation metrics verified against the live MLflow run.

Screenshot 8 - Docker database. We checked SQLite inside the registry container without changing it. The screenshot shows the file, tables, number of model versions and champion version 17. The

database file, /app/docker-mlflow.db, is kept in the shared project folder. SQLite runs inside the registry; it is not another container.

Docker registry: persistent SQLite database (read-only inspection)

```
{
  "database": "/app/docker-mlflow.db",
  "size_bytes": 1437696,
  "tables": [
    "experiments",
    "model_versions",
    "registered_model_aliases",
    "registered_models",
    "runs"
  ],
  "model_versions": 17,
  "champion_model_version": 17
}
```

9. Conclusion

Limitations

- We used one credit-card dataset. We cannot assume the same results for another bank or a different time period.
- Fraud patterns can change over time. The model would need to be checked again using recent labelled transactions.
- We have not added automatic retraining or checks for changes in model performance or data patterns. Retraining is manual. We have prediction logs and health checks, but they do not tell us whether the model is becoming less accurate.

In this assignment we built a fraud detection system with a pipe-and-filter training pipeline, an MLflow registry and a FastAPI microservice that serves the champion model, with prediction logging and automated tests. In the saved notebook run, 58 tests passed. With Docker Compose the registry and the prediction API also run as two separate containers. This serves the selected model as an assignment demonstration, not as an accepted production model. Against our goals: false-positive rate (1.91% against 2%) is met, latency (p95 27.8 ms against 200 ms) is met, and the test recall is 86.3% against the 90% target, so that goal is not met. The estimated fraud loss reduction is 82.4% against the 30% target (met). From this work we learned that with so few frauds in each split the recall on the test set can differ from the validation recall, that the model for serving should be chosen on validation results only, and that a registry alias keeps the service independent of the model version. If we had more time we would try more tuning on validation, cross-validation for the threshold, and more fraud examples.